C.E.S.A.R school

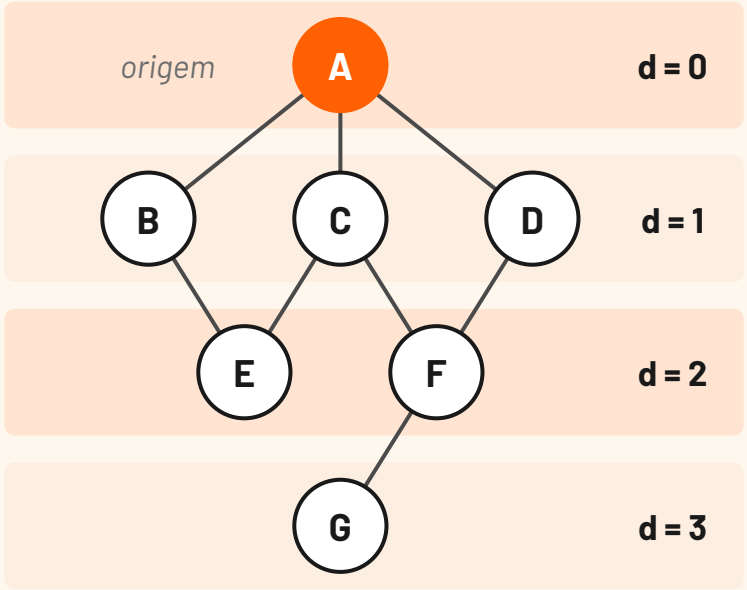
TEORIA DOS GRAFOS

Profª Laura Pacifico

2026 | OUTUBRO

Revisão: Busca em Largura (BFS)

- A partir da origem, visita **primeiro todos os vizinhos**, depois os vizinhos dos vizinhos, e assim por diante.
- Explora o grafo **em camadas**: a camada d reúne os vértices a d arestas da origem.
- Usa uma **fila** (FIFO): quem é descoberto primeiro é explorado primeiro.
- Em grafo sem pesos, encontra o **caminho com menos arestas** da origem até cada vértice.



Pseudocódigo: Busca em Largura *(BFS)*

Para cada vértice v, inicialize

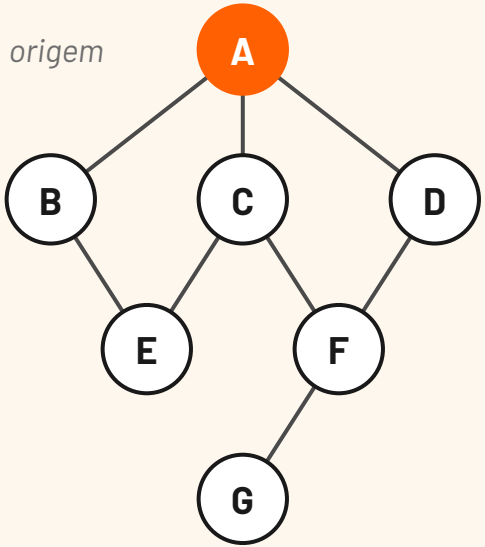
estado(v) = NAO_VISITADO
p(v) = nil
d(v) = ∞

Fila (FIFO): o primeiro a entrar é o primeiro a sair.

d(v): menor número de arestas de s até v.

```
BFS(s):
    estado(s) = VISITADO;  d(s) = 0
    Q = fila vazia;  insere s no fim de Q
    enquanto Q não estiver vazia:
        v_i = remove o início de Q
        para cada v_j adjacente a v_i:
            se estado(v_j) = NAO_VISITADO:
                estado(v_j) = VISITADO
                d(v_j) = d(v_i) + 1
                p(v_j) = v_i
                insere v_j no fim de Q
        estado(v_i) = ENCERRADO
```

Revisão: BFS passo a passo



Passo	Sai da fila	Entra na fila	Fila (início → fim)
1	A	B, C, D	B C D
2	B	E	C D E
3	C	F (E já está)	D E F
4	D	— (F já está)	E F
5	E	—	F
6	F	G	G
7	G	—	vazia

Ordem de visita: A, B, C, D, E, F, G

BFS × DFS

BFS: Busca em Largura

Estrutura
fila (FIFO)

Como explora
por camadas, a partir da origem

Garante
menor número de arestas até cada vértice

No grafo A–G
A, B, C, D, E, F, G

DFS: Busca em Profundidade

Estrutura
pilha ou recursão (LIFO)

Como explora
até o fim de um caminho, depois volta

Serve para
labirintos, ciclos, componentes

No grafo A–G
é o que veremos hoje

E se, em vez de terminar cada camada, seguíssemos o primeiro caminho até o fim antes de voltar?

Busca em Profundidade (Depth First Search - DFS)

- Originalmente idealizado pelo matemático francês Pierre Trémaux (1818-1895)

- A partir de um determinado nó, seleciona um determinado vizinho e atravessa completamente esse caminho antes de fazer o backtracking

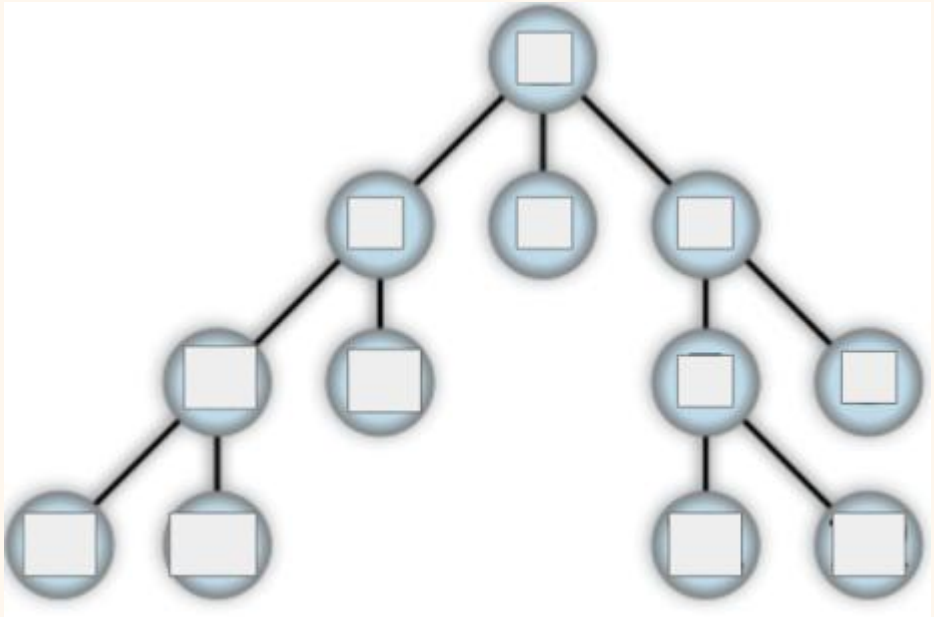
- Oferece uma ferramenta para simular possíveis caminhos para modelagem de cenários
    - Foi pensado originalmente para resolver o problema de sair de um labirinto

Busca em Profundidade (Depth First Search - DFS)

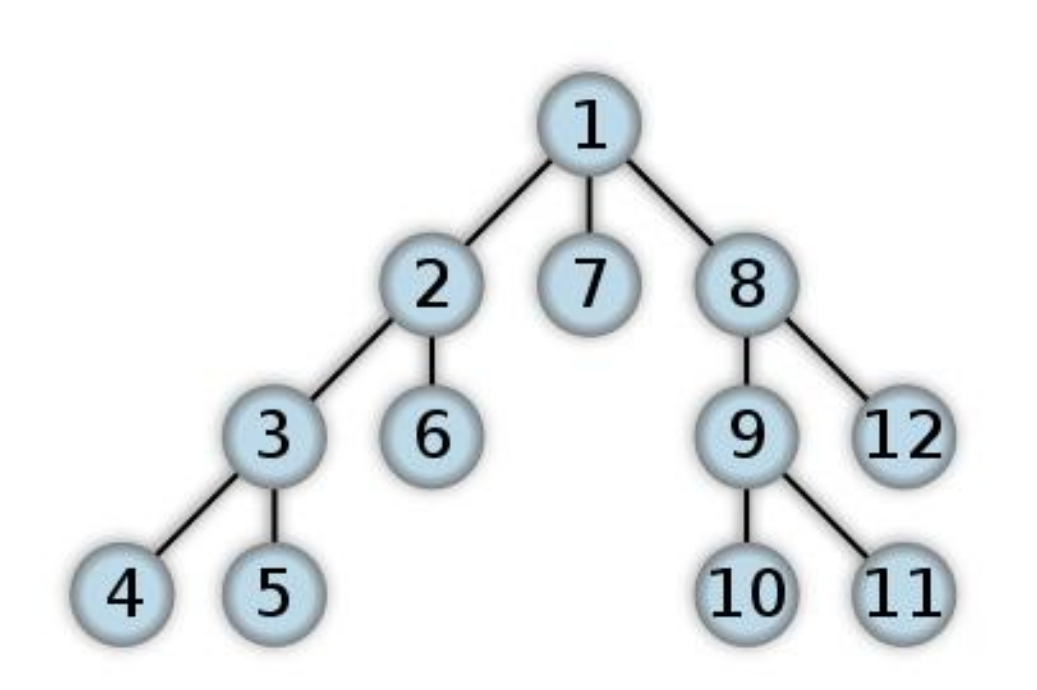
- A Busca em Profundidade (ou do inglês *Depth-First Search - DFS*) é um algoritmo ou estratégia que pode ser utilizada para percorrer estruturas como árvores e grafos.

- Em grafos, o algoritmo começa a partir de um vértice e explora a maior quantidade possível de vértices (navegando pelas arestas) usando a seguinte regra: a partir do primeiro vértice (potencialmente um vértice arbitrário) visita-se, recursivamente, seus vizinhos.

Busca em Profundidade (Depth First Search - DFS)



Busca em Profundidade (Depth First Search - DFS)



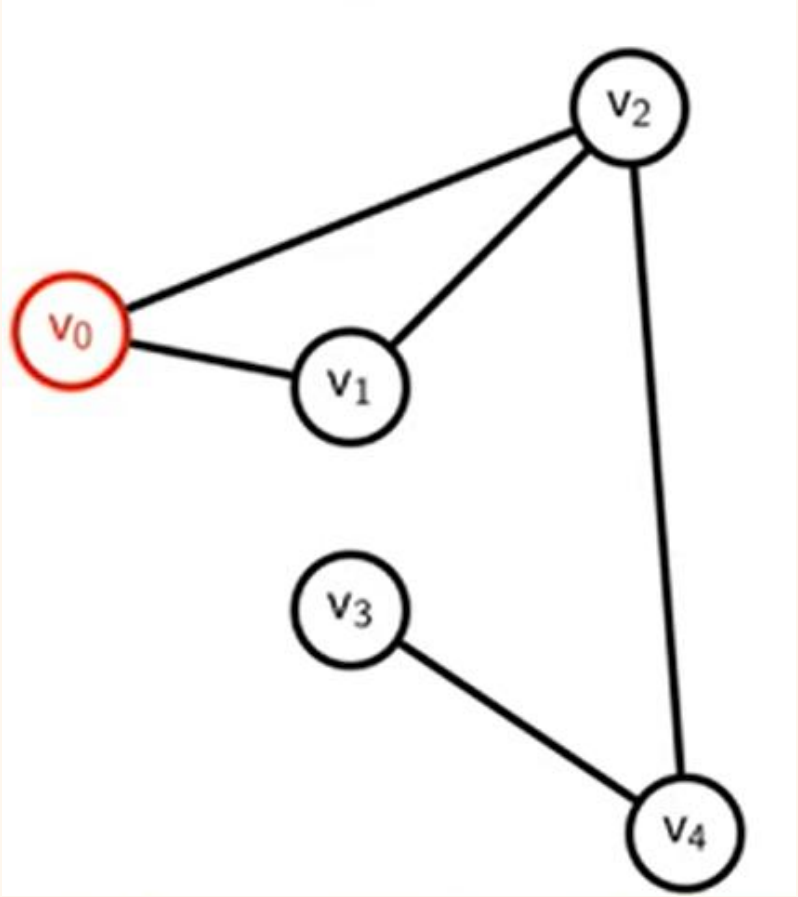
Pseudocódigo: Busca em Profundidade *(DFS)*

Para cada vértice, inicialize
estado(v) = NAO_VISITADO
p(v) = nil
tempo = -1

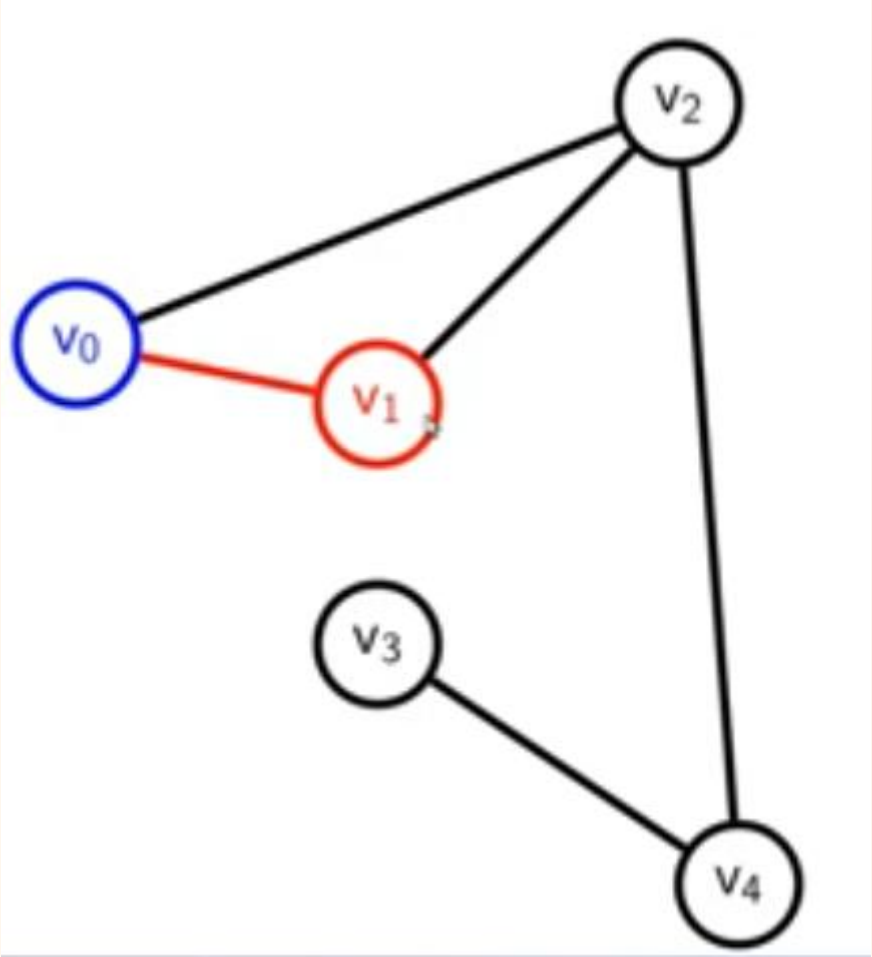
v_0 -> vértice inicial
estado(v) -> VISITADO, NAO_VISITADO, ENCERRADO
p(v) -> vértice pai do vetor
t(v) -> tempo para v0 ser encerrado

```
VisitaVertice(v_i):
    tempo = tempo + 1
    estado(v_i) = VISITADO
    t(v_i) = tempo
    para cada v_j adjacente a v_i :
        se estado(v_j) = NAO_VISITADO
            estado (v) = VISITADO
            p(v_j) = v_i
                VisitaVertice(v_j)
    estado(v_i) = ENCERRADO
    tempo = tempo + 1
```

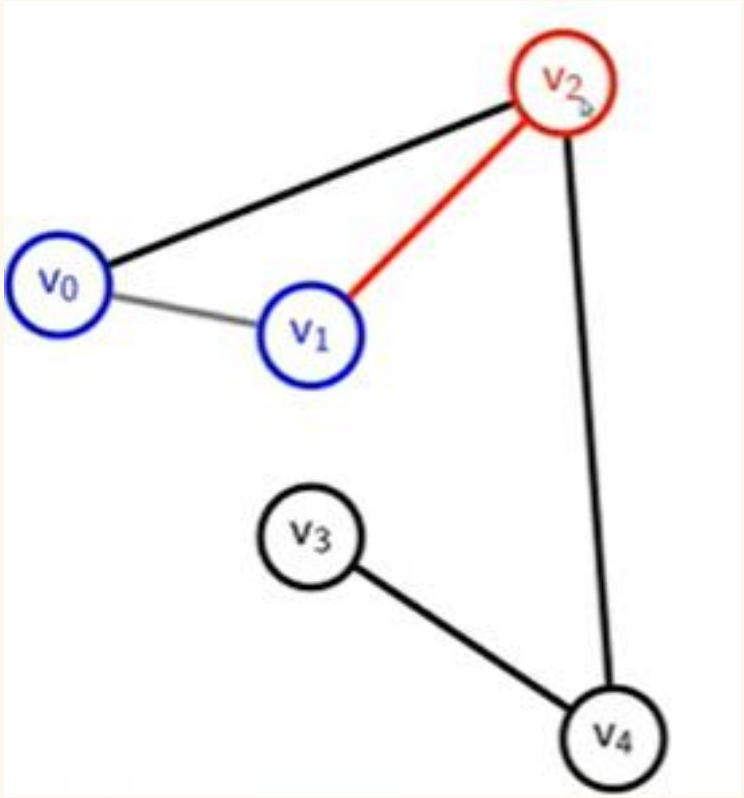
Busca em Profundidade



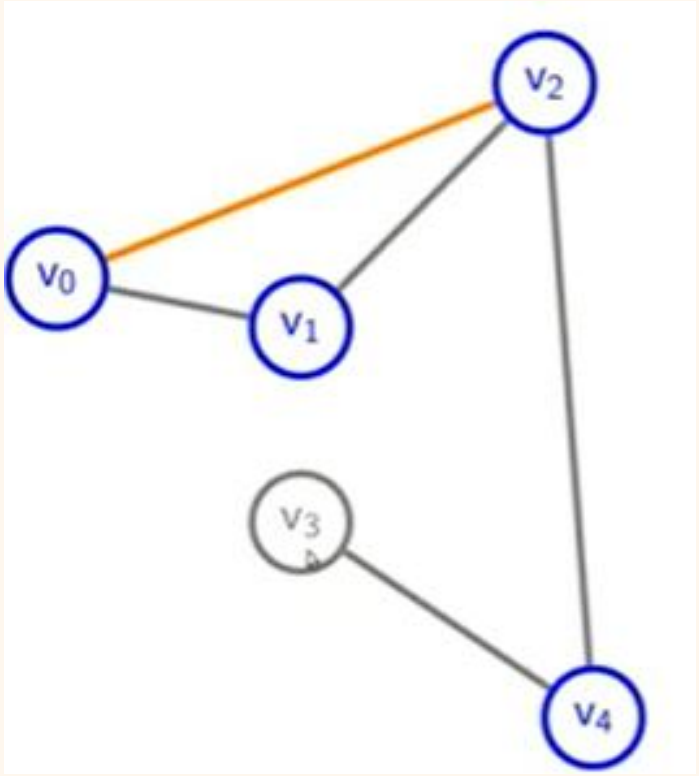
Busca em Profundidade



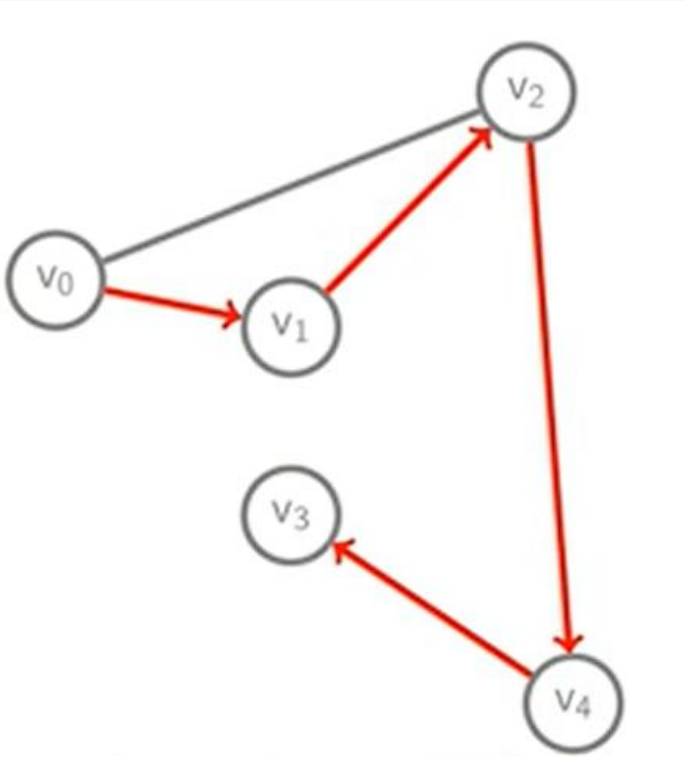
Busca em Profundidade



Busca em Profundidade

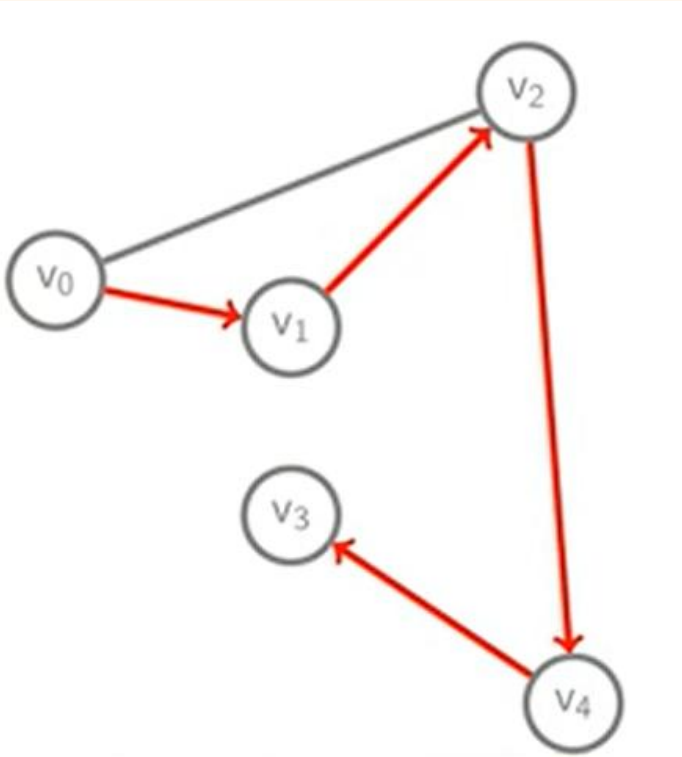


Busca em Profundidade



O caminho percorrido durante a busca ao visitar os vértices forma árvores conhecidas como árvores de busca em profundidade.

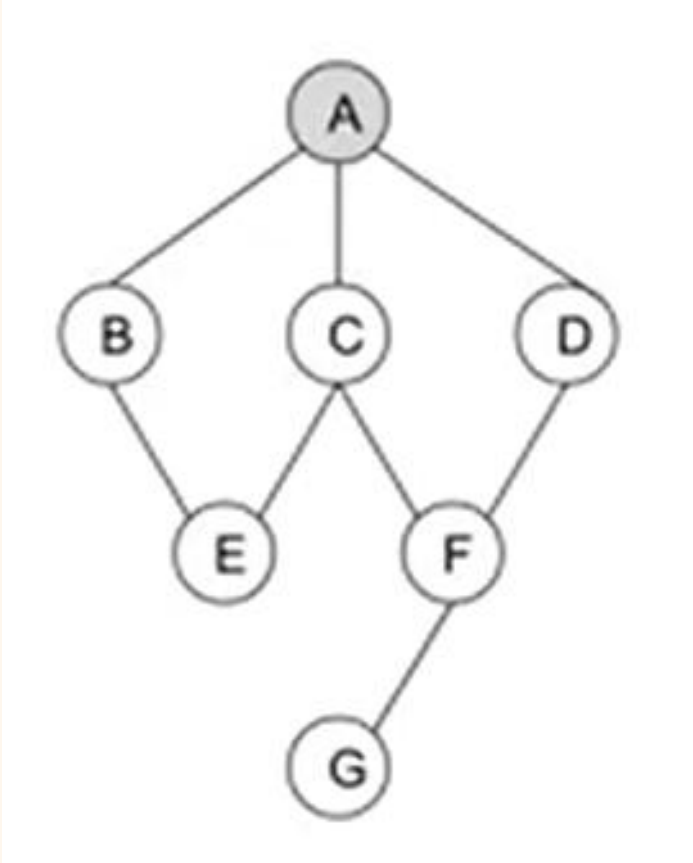
Busca em Profundidade



O caminho percorrido durante a busca ao visitar os vértices forma árvores conhecidas como árvores de busca em profundidade.

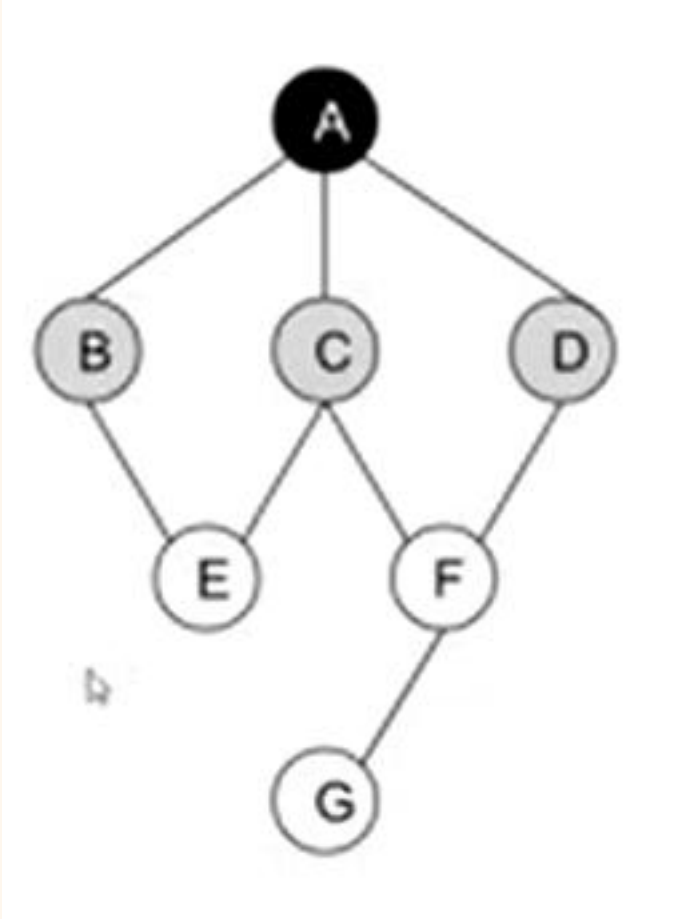
Observação: Nem sempre é possível, a partir de um único vértice, percorrer o grafo todo usando Busca em Profundidade (não será possível, por exemplo, em um grafo não direcionado que não seja conexo).

Busca em Profundidade



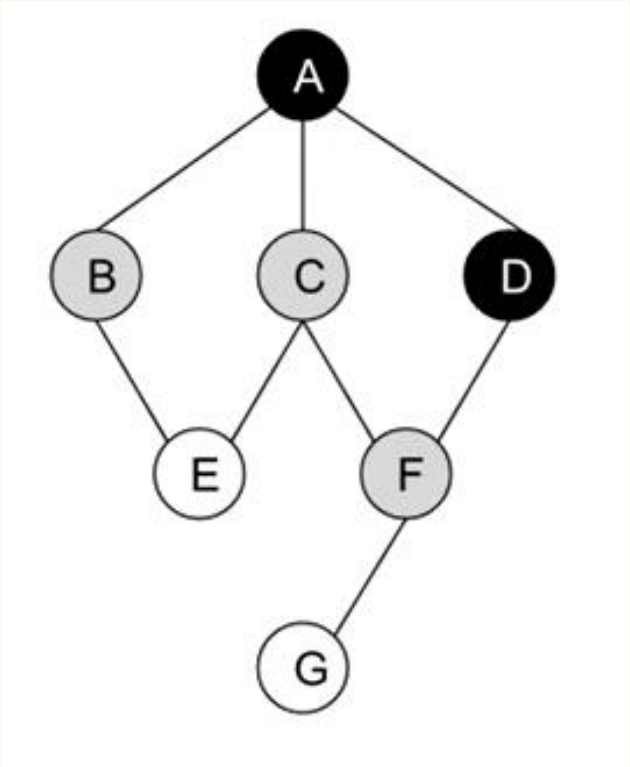
A

Busca em Profundidade



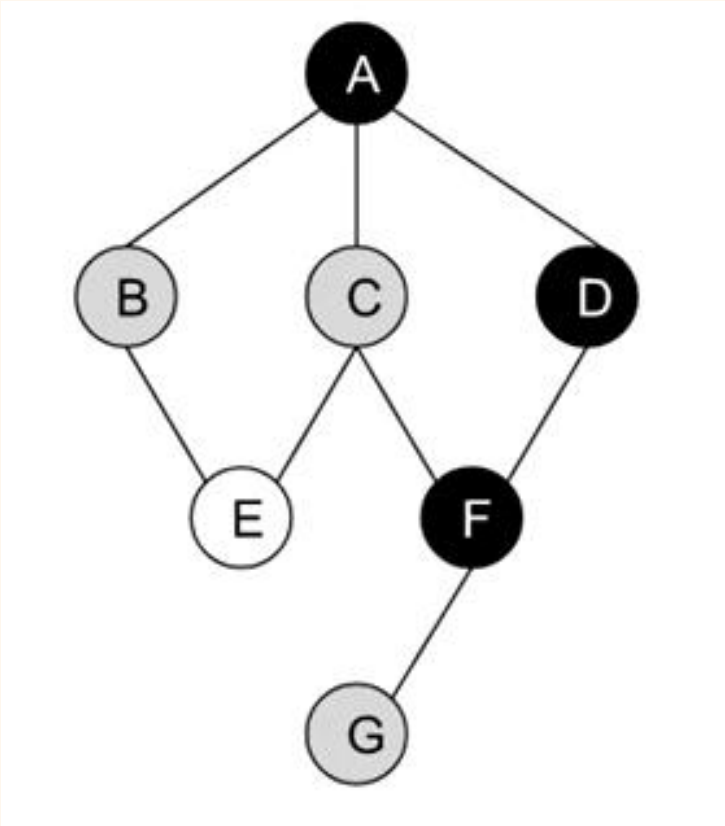
D
C
B

Busca em Profundidade



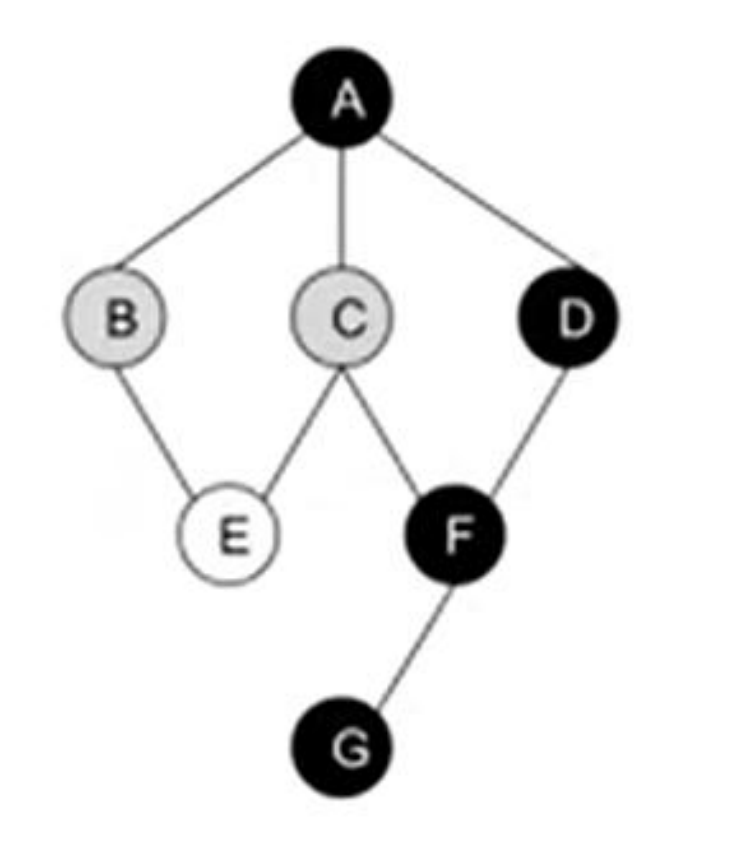
D
C
B

Busca em Profundidade



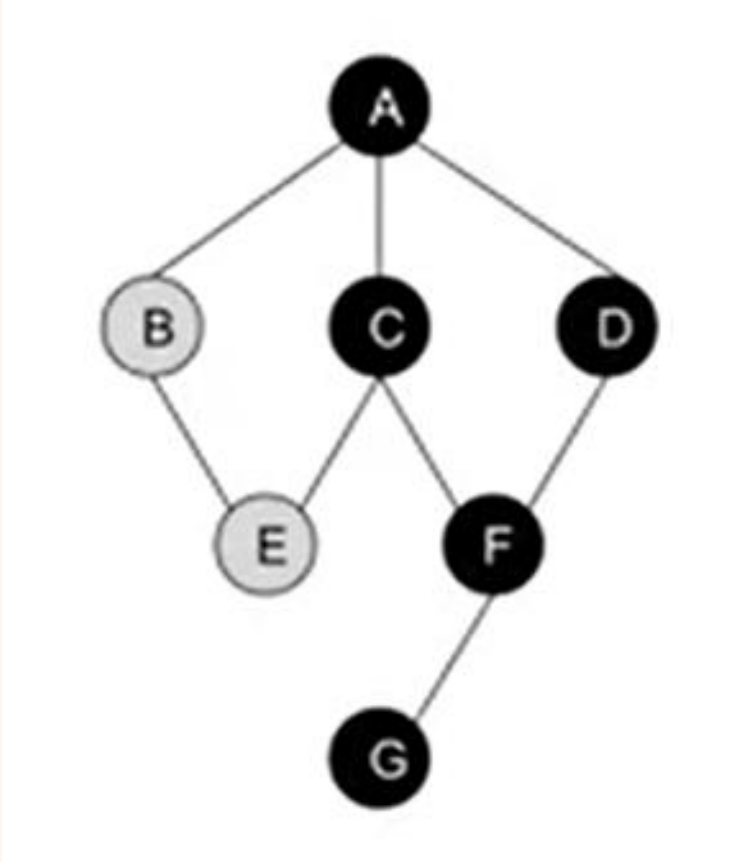
F
C
B

Busca em Profundidade



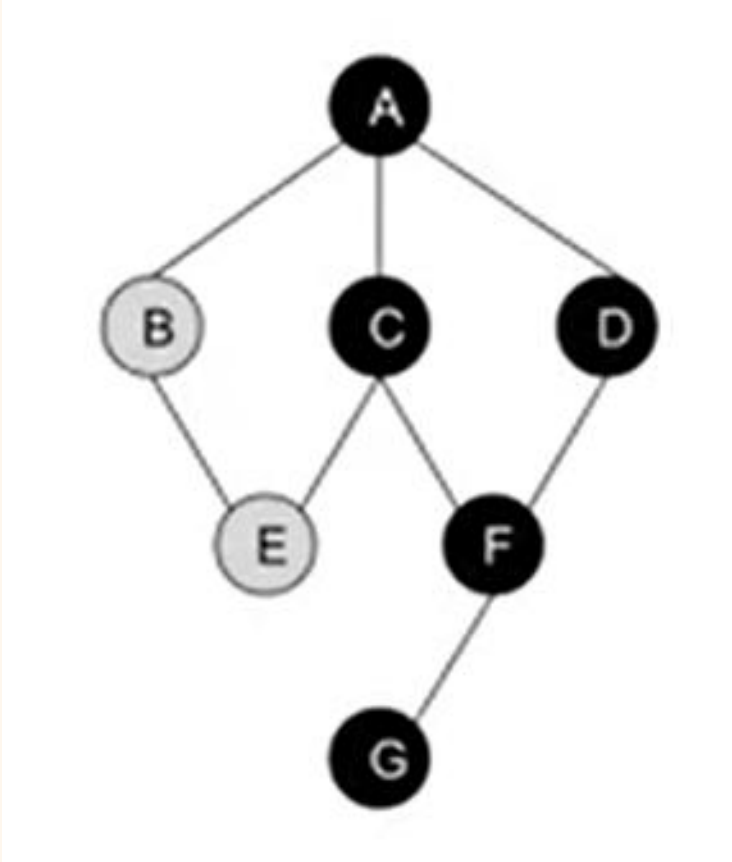
G
C
B

Busca em Profundidade



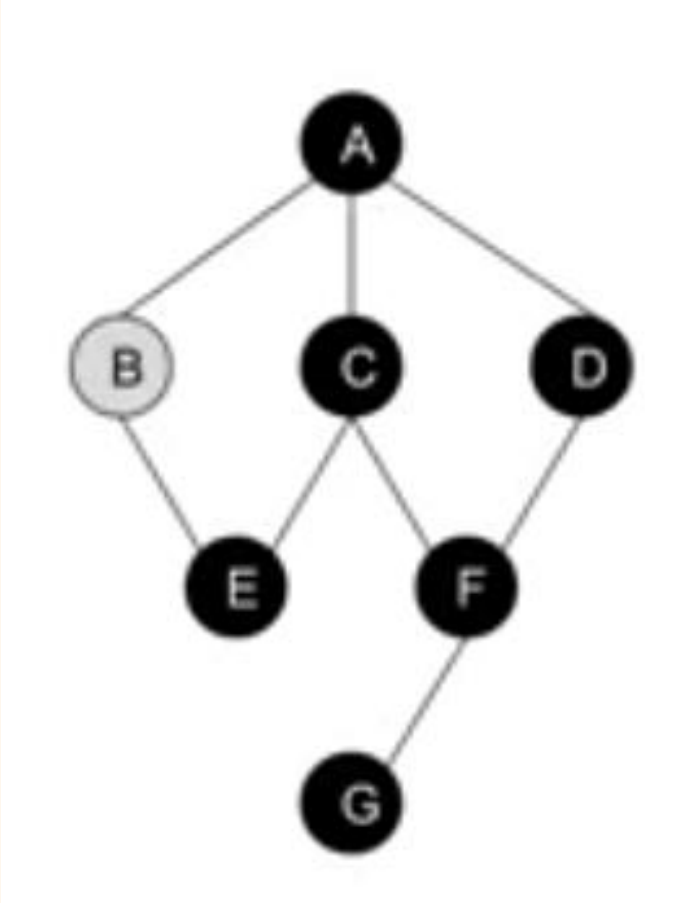
C
B

Busca em Profundidade



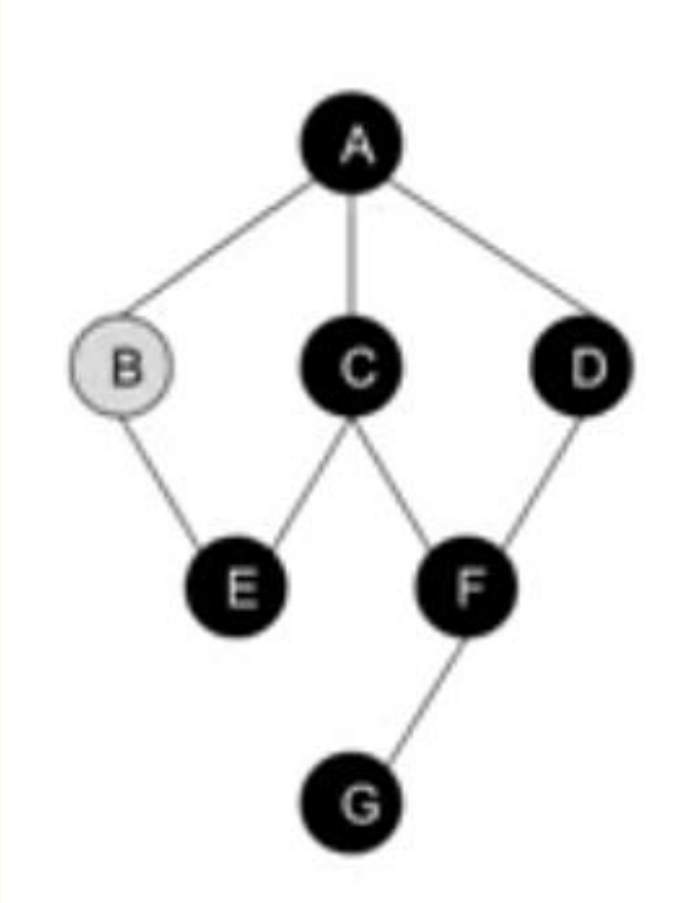
E
B

Busca em Profundidade



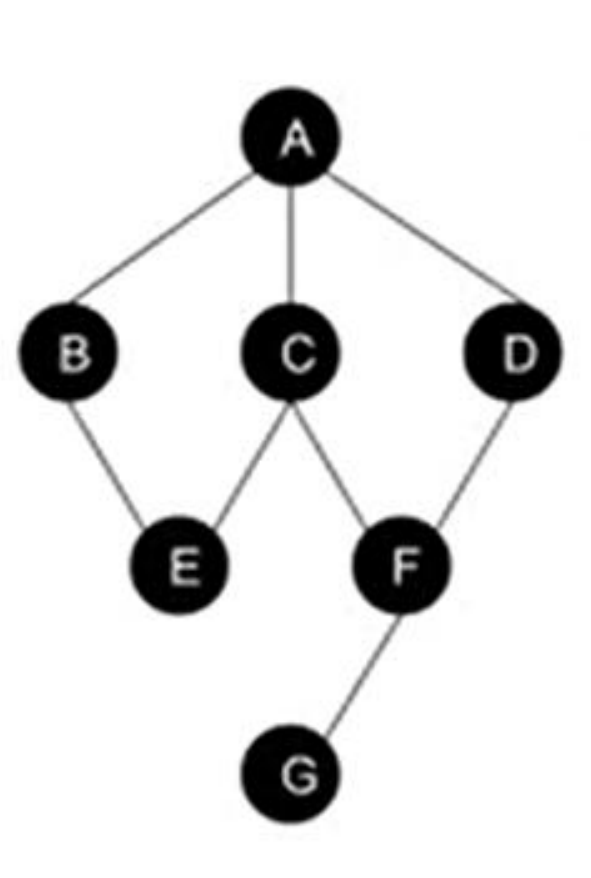
E
B

Busca em Profundidade



B

Busca em Profundidade



B

Busca em Profundidade

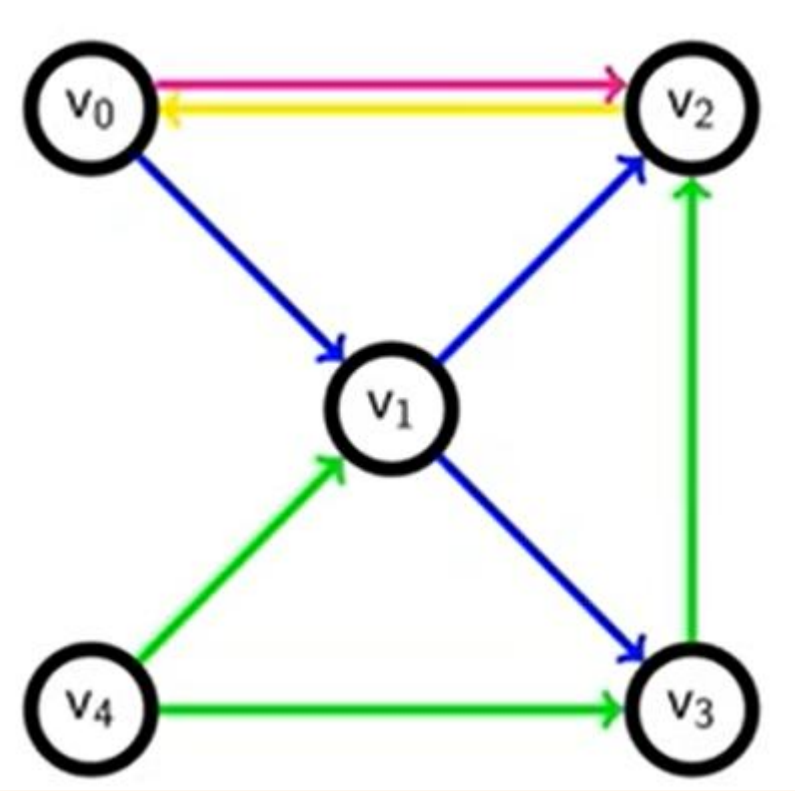
Consideraremos quatro tipos de arestas:

- **Arestas de árvore**: arestas de uma árvore de busca em profundidade (arestas utilizadas para acessar vértices pela primeira vez);
- **Arestas de retorno**: conectam um vértice v2 a um antecessor v1 em uma árvore de busca em profundidade;
- **Arestas de avanço**: não pertencem à árvore de busca em profundidade, mas conectam um vértice v1 a um sucessor v2 na árvore;
- **Arestas de cruzamento**: conectam vértices de árvores diferentes de busca ou da mesma árvore, mas que não sejam nem antecessores nem sucessores (por exemplo, irmãos).

Busca em Profundidade

Consideraremos quatro tipos de arestas:

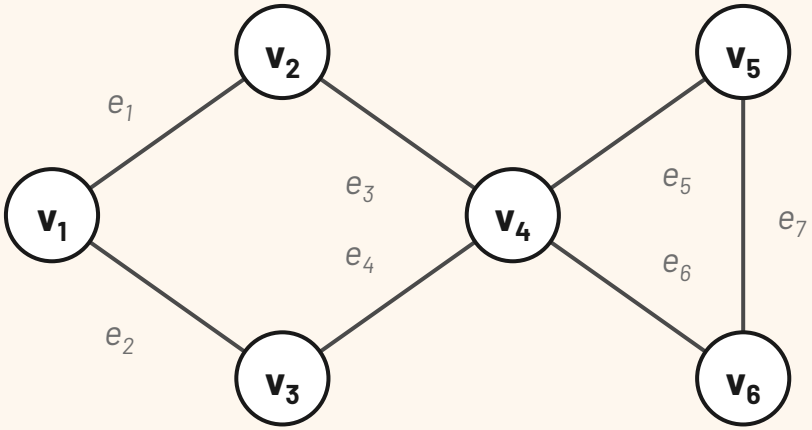
- **Arestas de árvore**: arestas de uma árvore de busca em profundidade (arestas utilizadas para acessar vértices pela primeira vez);
- **Arestas de retorno**: conectam um vértice v2 a um antecessor v1 em uma árvore de busca em profundidade;
- **Arestas de avanço**: não pertencem à árvore de busca em profundidade, mas conectam um vértice v1 a um sucessor v2 na árvore;
- **Arestas de cruzamento**: conectam vértices de árvores diferentes de busca ou da mesma árvore, mas que não sejam nem antecessores nem sucessores (por exemplo, irmãos).



Arestas de árvore
Arestas de retorno
Arestas de avanço
Arestas de cruzamento

Exercício guiado: vamos fazer juntos

Aplique a DFS a partir de $\mathbf{v_1}$, visitando os vizinhos em **ordem crescente**.



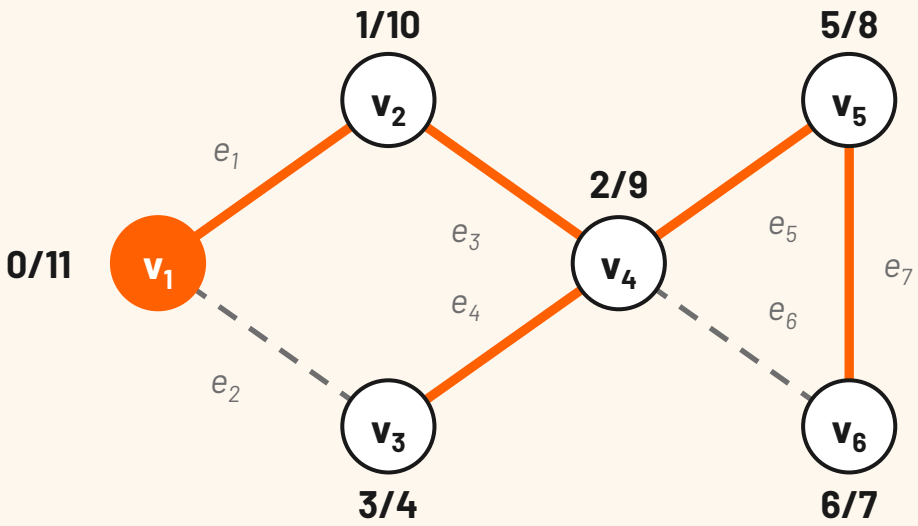
Anote, passo a passo:

1. a sequência de vértices visitados;
2. os tempos de descoberta d(v) e de término f(v);
3. quais arestas são de árvore e quais são de retorno;
4. para comparar: qual seria a sequência da BFS?

Exercício guiado: passo a passo

Passo	Em	Olha	O que acontece	Pilha de chamadas
1	v_1	—	descobre v_1, d = 0	v_1
2	v_1	v_2	novo ⟶ desce para v_2, d = 1	v_1 v_2
3	v_2	v_4	v_1 é o pai; v_4 é novo ⟶ desce, d = 2	v_1 v_2 v_4
4	v_4	v_3	v_2 é o pai; v_3 é novo ⟶ desce, d = 3	v_1 v_2 v_4 v_3
5	v_3	v_1	v_1 ainda aberto ⟶ retorno (e_2); v_3 encerra, f = 4	v_1 v_2 v_4
6	v_4	v_5	novo ⟶ desce, d = 5	v_1 v_2 v_4 v_5
7	v_5	v_6	novo ⟶ desce, d = 6	v_1 v_2 v_4 v_5 v_6
8	v_6	v_4	v_4 ainda aberto ⟶ retorno (e_6); v_6 encerra, f = 7	v_1 v_2 v_4 v_5
9	—	—	volta: v_5, v_4, v_2, v_1 encerram, f = 8, 9, 10, 11	vazia

Exercício guiado: resolução



aresta de árvore — — aresta de retorno **d/f** descoberta/término

Sequência DFS

v_1 v_2 v_4 v_3 v_5 v_6

Arestas de árvore

e_1, e_3, e_4, e_5, e_7

Arestas de retorno

$e_2\,(v_3 \to v_1)$ e $e_6\,(v_6 \to v_4)$

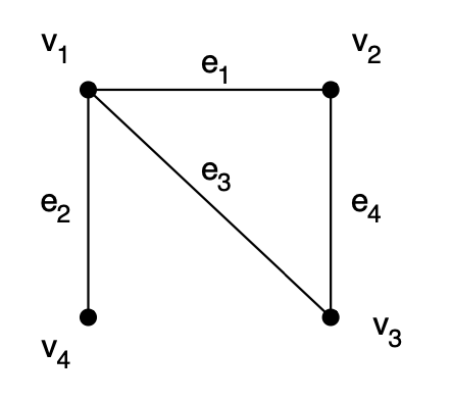
BFS, para comparar

v_1 v_2 v_3 v_4 v_5 v_6

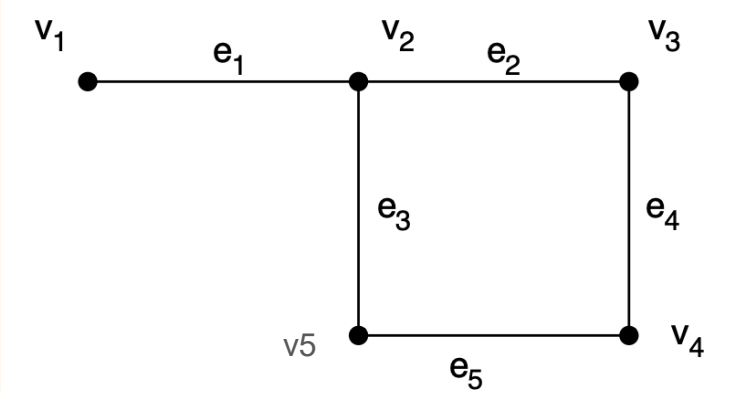
Repare: v_3 é vizinho de v_1, mas a DFS só chega nele depois de descer por v_2 e v_4.

Busca em Profundidade

Apresente a sequência de vértices após a aplicação da DFS a partir do vértice.



v3



v1

Dúvidas?



Laura Alves Pacifico
laps@cesar.school
Slack: Laura Pacifico